



CYBERSTER BLUE TEAM (SOC) INTERNSHIP

Week 1 — SOC Operations & Lab Setup

Name: Haris Zamir

Roll Number: CSI-B1-487

Program: Cyberster Blue Team (SOC) Internship

Instructor: Abdullah Zia

Task Timeline: Week 1

1. Introduction

A Security Operations Center (SOC) serves as the nerve center for monitoring, detecting, and responding to cybersecurity threats across an organization's infrastructure. SOC analysts work around the clock, leveraging SIEM (Security Information and Event Management) platforms to collect, correlate, and analyze log data from various endpoints and network devices.

This Week 1 report documents the foundational setup of a personal SOC lab environment built to simulate real-world threat detection scenarios. The lab was designed to mirror a minimal enterprise network where a SIEM platform actively monitors endpoints for suspicious activity.

Lab Environment Overview

The lab consisted of three virtual machines running on Oracle VirtualBox:

- Ubuntu Server (22.04 LTS) — Hosted the Wazuh SIEM platform (Manager, Indexer, and Dashboard)
- Windows 11 — A monitored Windows endpoint with the Wazuh agent installed
- Kali Linux — Used as an attacker simulation machine for generating test alerts

By the end of Week 1, the lab environment was fully operational. The Wazuh SIEM platform was successfully generating and displaying security alerts triggered by simulated attack activity from the Kali Linux machine. This report details the complete journey — from understanding core networking fundamentals to deploying agents, triaging alerts, and mapping events to the MITRE ATT&CK framework.

Day 1 — Networking Fundamentals for SOC

Before building any lab environment, it is essential to understand the underlying networking concepts that SOC analysts encounter daily. Network traffic analysis, protocol identification, and understanding how packets flow across systems are foundational skills. This day was dedicated to studying these concepts and applying them practically with Wireshark.

The OSI Model

The OSI (Open Systems Interconnection) Model is a conceptual framework that standardizes how different network systems communicate. Developed by the International Organization for Standardization, it divides network communication into seven distinct layers, each responsible for a specific function. For a SOC analyst, the OSI model provides a mental map for diagnosing where in the communication stack an anomaly or attack originates.

Layer	Name	Key Protocols / Role	SOC Relevance
7	Application	HTTP, DNS, HTTPS, SSH, RDP	Most alerts originate here — malware C2, web attacks
6	Presentation	TLS/SSL, Encryption, Compression	TLS inspection for encrypted traffic analysis
5	Session	NetBIOS, RPC, Session management	Session hijacking detection

Layer	Name	Key Protocols / Role	SOC Relevance
4	Transport	TCP, UDP — port-level communication	Port scanning, brute-force, DoS detection
3	Network	IP, ICMP, Routing protocols	Network scanning, IP spoofing, routing anomalies
2	Data Link	Ethernet, MAC addresses, ARP	ARP spoofing, MAC flooding attacks
1	Physical	Cables, signals, hardware	Physical tapping, less visible in SOC

From a SOC analyst's perspective, the most operationally relevant layers are Layer 2 through Layer 7. These layers are where the majority of detectable attack behaviors manifest — from ARP poisoning at the Data Link layer to application-level exploits at Layer 7.

The TCP/IP Model

While the OSI model is an excellent theoretical framework, real-world networks operate on the TCP/IP model. This 4-layer model is what all monitoring tools, packet captures, and SIEM platforms actually work with. Understanding the TCP/IP model makes it easier to interpret alerts and analyze traffic.

- Network Interface Layer — Physical hardware and data framing (maps to OSI Layers 1-2)
- Internet Layer — IP addressing and packet routing (maps to OSI Layer 3)
- Transport Layer — TCP/UDP communication, port management (maps to OSI Layer 4)
- Application Layer — All user-facing protocols: HTTP, DNS, SSH, etc. (maps to OSI Layers 5-7)

Example: When browsing to a website, the browser creates an HTTP request (Application), TCP segments it with port 443 (Transport), IP routes it to the server (Internet), and the network adapter physically transmits it (Network Interface). A SOC analyst monitoring this would see DNS queries, TCP handshakes, and TLS sessions in Wireshark.

Subnetting and CIDR

Subnetting divides a large IP address space into smaller, manageable network segments. This is critical for SOC analysts because it helps identify which network segment an alert belongs to, detect lateral movement between subnets, and understand the logical layout of the monitored environment.

In this lab, the VirtualBox Host-Only network used the following subnet:

```
Network: 192.168.100.0/24
```

```
Usable Host Range: 192.168.100.1 – 192.168.100.254
```

All three VMs were assigned static IPs within this range to ensure consistent communication for monitoring and testing.

Common SOC Monitoring Ports

SOC analysts must be familiar with the ports that are most frequently targeted by attackers or used for suspicious activity:

Port	Protocol / Service	SOC Monitoring Purpose
22	SSH	Remote login — monitor for brute-force attempts
80	HTTP	Unencrypted web traffic — detect data exfiltration, malware C2
443	HTTPS	Encrypted web — monitor metadata, certificates, anomalies
53	DNS	Domain resolution — detect DNS tunneling, malicious domains
445	SMB	Windows file sharing — WannaCry, lateral movement
3389	RDP	Remote Desktop — brute-force, unauthorized remote access
1514/1515	Wazuh Agent	Log forwarding to Wazuh SIEM manager

Key Network Protocols

ARP (Address Resolution Protocol)

ARP resolves IP addresses to physical MAC addresses on a local network. Before any communication can occur on a LAN, the sender broadcasts an ARP request asking which device holds a given IP address. Attackers exploit this by performing ARP spoofing — sending fake ARP replies to redirect traffic through their machine.

DNS (Domain Name System)

DNS translates human-readable domain names into IP addresses. From a SOC perspective, DNS is one of the most important protocols to monitor because malware frequently uses it for command-and-control communication. Unusual DNS query patterns, queries to newly registered domains, or abnormally large DNS responses may indicate DNS tunneling or malware beaconing.

```
Example: google.com → 142.250.190.78
```

HTTP / HTTPS

HTTP transmits data in plaintext, making it easy to inspect but also susceptible to interception. HTTPS encrypts communication using TLS, protecting confidentiality. However, even with HTTPS, SOC analysts can still observe connection metadata: IP addresses, ports, certificate subjects, and traffic volume patterns.

ICMP

ICMP is primarily used for diagnostic purposes — ping tests confirm whether a host is reachable. However, attackers also use ICMP for network reconnaissance (ping sweeps) and occasionally for covert data exfiltration (ICMP tunneling). Monitoring for large volumes of ICMP traffic is good practice.

```
ping 8.8.8.8
```

Wireshark Traffic Capture

Wireshark was used to capture and analyze live network traffic. This hands-on exercise helped build the ability to identify normal traffic patterns — a critical skill for later recognizing abnormal or malicious activity.

ICMP

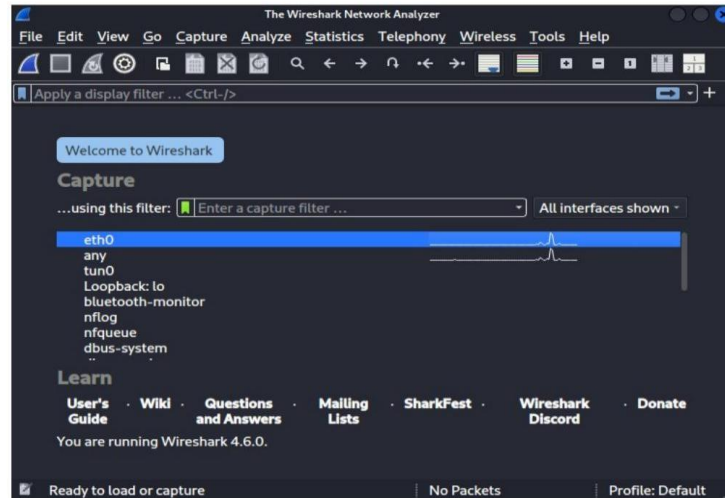
ICMP is used for diagnostics such as ping.

Example command:

```
Ping 8.8.8.8
```

Wireshark Traffic Capture

Wireshark was used to capture live traffic while browsing normally.



Several protocols were identified during packet analysis:

- DNS queries
- TCP connections

7

Figure 1: Wireshark interface showing available network interfaces for capture

During the capture session, the following protocols were identified and analyzed:

- DNS — Queries and responses for domain name resolution
- TCP — Three-way handshakes establishing connections
- ICMP — Ping echo requests and replies
- TLS/QUIC — Encrypted HTTPS sessions
- ARP — IP-to-MAC address resolution on the local network

- ICMP packets
- TLS sessions

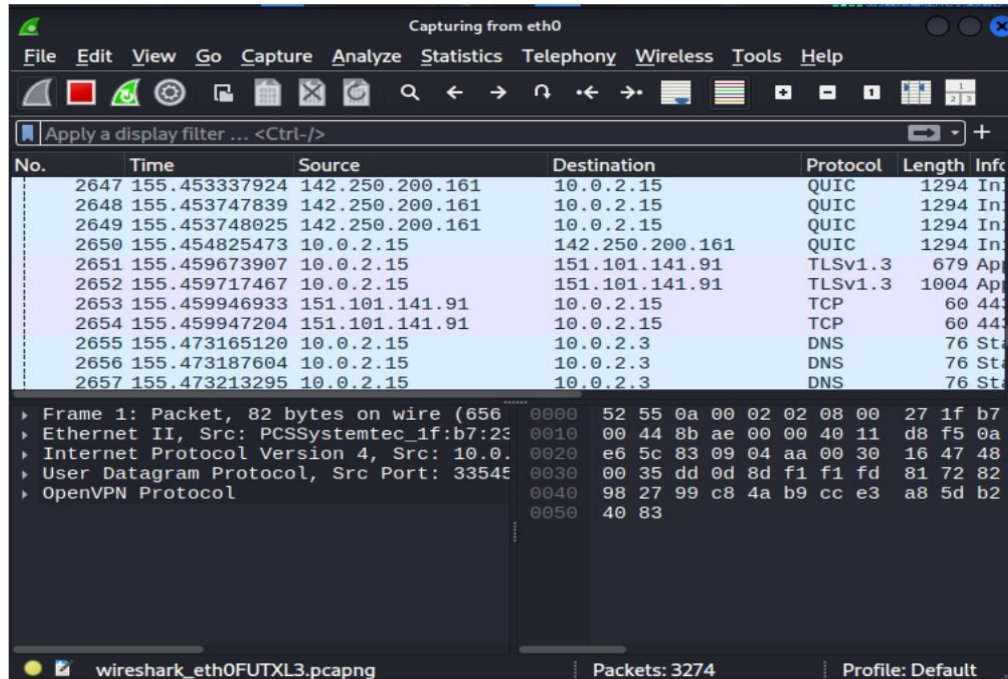


Figure 2: Live Wireshark capture showing DNS, QUIC, TCP, and TLS traffic

Studying normal traffic patterns through Wireshark is foundational for SOC work. An analyst who understands what legitimate DNS, ICMP, and TLS traffic looks like will be far better equipped to spot deviations that indicate scanning, exfiltration, or command-and-control communication.

Day 1 Deliverables

- Wireshark capture file containing network traffic from a live browsing session
- Identification and documentation of five or more protocols from the capture
- Written summary of networking concepts and their SOC relevance

Day 2 — VirtualBox Lab Setup

With the networking fundamentals in place, the next step was to build the physical infrastructure of the SOC lab. Oracle VirtualBox was used to create and configure three virtual machines that would simulate a small monitored network environment.

Virtual Machines Created

Machine	Operating System	Role in Lab
Ubuntu Server	Ubuntu 22.04 LTS	Wazuh SIEM platform (Manager, Indexer, Dashboard)
Windows 11	Windows 11 Enterprise	Monitored endpoint — Wazuh agent installed
Kali Linux	Kali Linux 2025.3	Attacker simulation and penetration testing

Resource Allocation

Wazuh's all-in-one installation is resource-intensive, particularly due to the Wazuh Indexer which runs on Java. The Ubuntu Server was therefore given more RAM to ensure stable operation:

Machine	CPU Cores	RAM	Storage
Ubuntu Server	4 cores	6 GB	100 GB
Windows 11	4 cores	4 GB	80 GB
Kali Linux	4 cores	4 GB	100 GB

VirtualBox Network Configuration

Each virtual machine was configured with two network adapters to satisfy different connectivity requirements:

- Adapter 1 — NAT: Provides internet access for downloading packages and updates
- Adapter 2 — Host-Only: Enables private internal communication between all VMs and the host

The Host-Only adapter is critical for the SOC lab because it creates an isolated internal network. The Wazuh agents on the endpoint VMs communicate with the Wazuh Manager exclusively through this Host-Only network. This prevents unintended exposure to the broader internet while still allowing the VMs to download updates via NAT.

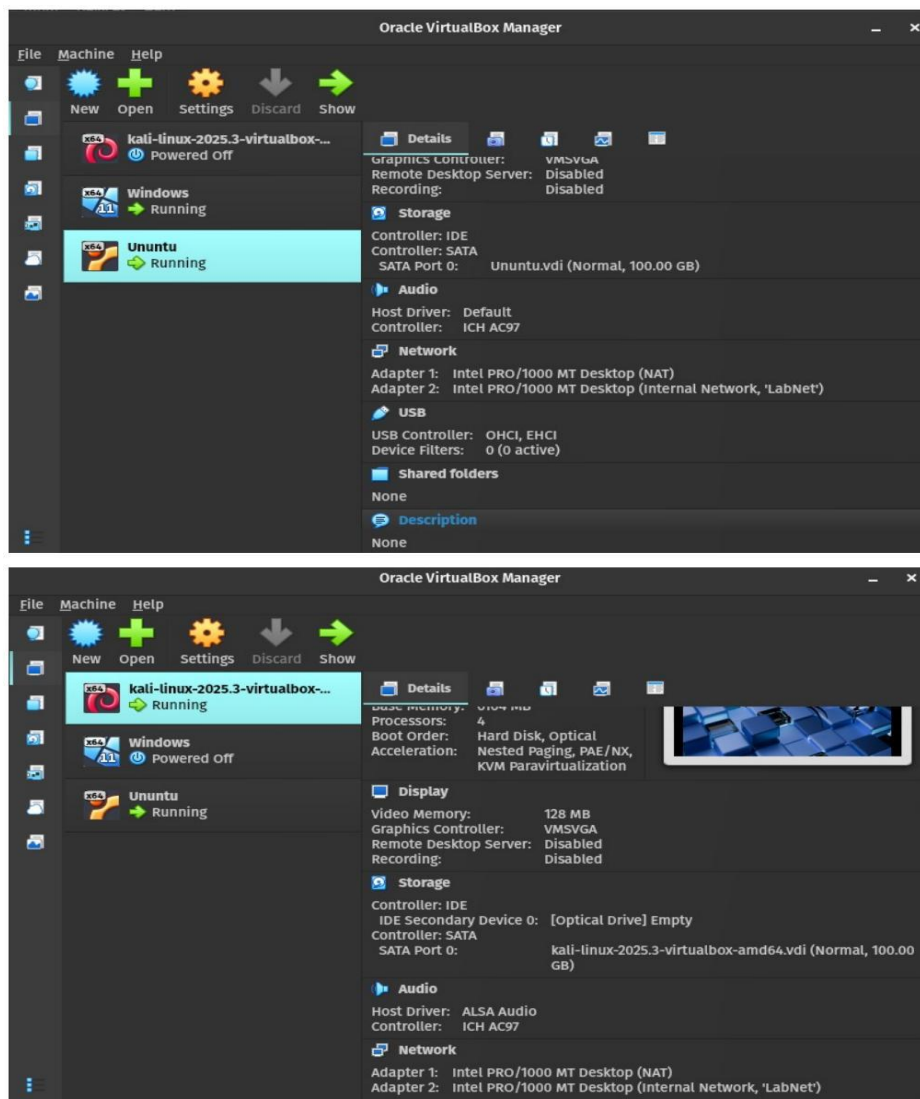


Figure 3: Oracle VirtualBox Manager showing all three VMs with network adapter configuration

Checking IP Addresses

After configuring the network adapters, static IP addresses were verified on each machine. Each VM had two active network interfaces — one for NAT (internet) and one for the Host-Only internal network. The following addresses were confirmed:

Virtual Machine	Host-Only IP Address	Role
Ubuntu Server (Wazuh)	192.168.100.10	SIEM Manager — receives logs from all agents
Kali Linux	192.168.100.20	Attacker — generates test alerts
Windows 11	192.168.100.30	Monitored endpoint — sends logs to Wazuh

```

C:\Windows\System32>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : fd17:625c:f037:2:f0ec:55db:c00f:4de9
    Temporary IPv6 Address. . . . . : fd17:625c:f037:2:e874:fc95:bc35:ce29
    Link-local IPv6 Address . . . . . : fe80::b5a:39a4:a0a:d8ad%10
    IPv4 Address. . . . . : 10.0.2.15
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::2%10
                             10.0.2.2

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::e8dd:c511:f9a0:f981%13
    IPv4 Address. . . . . : 192.168.100.20
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.100.10

C:\Windows\System32>_

```

```

valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:6b:4c:ac brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 58839sec preferred_lft 58839sec
    inet 192.168.100.10/24 brd 192.168.100.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fd17:625c:f037:2:7a6c:8f67:b354:9c66/64 scope global temporary dynamic
        valid_lft 86155sec preferred_lft 14155sec
    inet6 fd17:625c:f037:2:a00:27ff:fe6b:4cac/64 scope global dynamic mngtmpaddr
        valid_lft 86155sec preferred_lft 14155sec
    inet6 fe80::a00:27ff:fe6b:4cac/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:a2:fd:1f brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.10/24 scope global enp0s8
        valid_lft forever preferred_lft forever

```

Figure 4: IP configuration output showing both NAT and Host-Only interfaces on Windows and Ubuntu

Connectivity Testing

After assigning IP addresses, ping tests were performed between all three machines to confirm bidirectional connectivity through the Host-Only network. All tests returned successful results with zero packet loss, confirming that the network was correctly configured.

```

ping 192.168.100.10    # From Kali to Ubuntu (Wazuh Manager)
ping 192.168.100.20    # From Ubuntu to Kali
ping 192.168.100.30    # From Ubuntu to Windows

```

```
haris@hariszamir-VirtualBox:~$ ping 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.
64 bytes from 192.168.100.20: icmp_seq=1 ttl=128 time=1.35 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=128 time=1.15 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=128 time=1.67 ms
64 bytes from 192.168.100.20: icmp_seq=4 ttl=128 time=0.505 ms
^C
--- 192.168.100.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 0.505/1.169/1.669/0.425 ms
```

```
haris@hariszamir-VirtualBox:~$ ping 192.168.100.30
PING 192.168.100.30 (192.168.100.30) 56(84) bytes of data.
64 bytes from 192.168.100.30: icmp_seq=1 ttl=64 time=1.35 ms
64 bytes from 192.168.100.30: icmp_seq=2 ttl=64 time=0.477 ms
64 bytes from 192.168.100.30: icmp_seq=3 ttl=64 time=0.566 ms
64 bytes from 192.168.100.30: icmp_seq=4 ttl=64 time=0.545 ms
^C
--- 192.168.100.30 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3081ms
rtt min/avg/max/mdev = 0.477/0.734/1.351/0.357 ms
```

```
C:\Windows\System32>ping 192.168.100.20

Pinging 192.168.100.20 with 32 bytes of data:
Reply from 192.168.100.20: bytes=32 time<1ms TTL=128
Reply from 192.168.100.20: bytes=32 time<1ms TTL=128
Reply from 192.168.100.20: bytes=32 time<1ms TTL=128
Reply from 192.168.100.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.100.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Windows\System32>
```

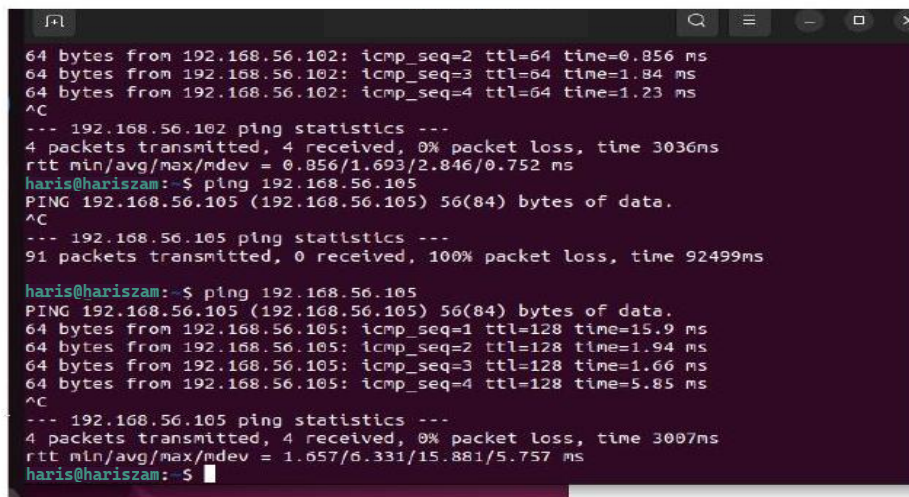
Figure 5: Successful ping tests confirming connectivity across all VMs in the Host-Only network

Step 3 - Verifying connectivity between all VMs

To ensure the lab network is properly configured, we tested connectivity between all virtual machines (VMs) on the host-only network (192.168.56.x). The VMs include Kali Linux, Ubuntu, and Windows. Each VM also has a NAT adapter for internet access.

1. Ubuntu ↔ Windows

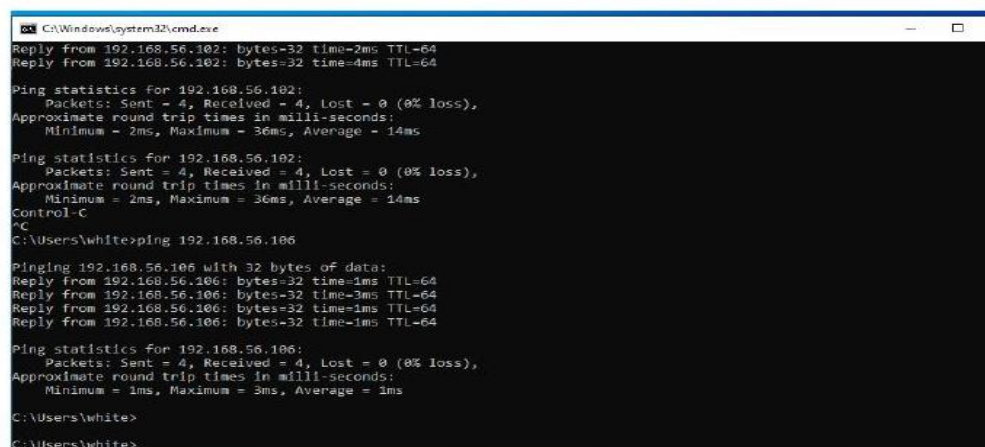
- Ubuntu was able to ping Windows successfully after enabling ICMP (ping) on the Windows firewall.



```
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.856 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=1.84 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=1.23 ms
^C
--- 192.168.56.102 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.856/1.693/2.846/0.752 ms
haris@hariszam:~$ ping 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data.
^C
--- 192.168.56.105 ping statistics ---
91 packets transmitted, 0 received, 100% packet loss, time 92499ms

haris@hariszam:~$ ping 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data.
64 bytes from 192.168.56.105: icmp_seq=1 ttl=128 time=15.9 ms
64 bytes from 192.168.56.105: icmp_seq=2 ttl=128 time=1.94 ms
64 bytes from 192.168.56.105: icmp_seq=3 ttl=128 time=1.66 ms
64 bytes from 192.168.56.105: icmp_seq=4 ttl=128 time=5.85 ms
^C
--- 192.168.56.105 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.657/6.331/15.881/5.757 ms
haris@hariszam:~$
```

- Windows can ping Ubuntu without issues, confirming bidirectional communication.



```
C:\Windows\system32\cmd.exe
Reply from 192.168.56.102: bytes=32 time=2ms TTL=64
Reply from 192.168.56.102: bytes=32 time=4ms TTL=64

Ping statistics for 192.168.56.102:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 36ms, Average = 14ms

Ping statistics for 192.168.56.102:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 36ms, Average = 14ms
Control-C
^C
C:\Users\white>ping 192.168.56.106

Pinging 192.168.56.106 with 32 bytes of data:
Reply from 192.168.56.106: bytes=32 time=1ms TTL=64
Reply from 192.168.56.106: bytes=32 time=3ms TTL=64
Reply from 192.168.56.106: bytes=32 time=1ms TTL=64
Reply from 192.168.56.106: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.56.106:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 3ms, Average = 1ms

C:\Users\white>
C:\Users\white>
```

Figure 6: Bidirectional ping tests between Ubuntu and Windows confirming full network connectivity

Day 2 Deliverables

- Screenshots of VirtualBox network adapter configuration for all three VMs
- Screenshots confirming IP address assignment on each VM
- Ping test screenshots confirming full bidirectional connectivity

Days 3–4 — Wazuh Manager Installation

Wazuh is a free, open-source SIEM and XDR platform that provides threat detection, integrity monitoring, and compliance capabilities. It was chosen for this lab because of its comprehensive feature set, active community, and its ability to deploy agents across multiple operating systems — making it ideal for simulating a real SOC environment.

Wazuh Architecture

The Wazuh platform consists of three core components that must be installed and running for the system to function:

- Wazuh Manager — The central brain that analyzes logs from agents and generates alerts based on detection rules
- Wazuh Indexer — An OpenSearch-based data store that indexes all security events for fast search and retrieval
- Wazuh Dashboard — A web-based visualization interface for viewing alerts, managing agents, and performing threat hunting

These three components are collectively referred to as the Wazuh Stack. In this lab, all three were installed on the single Ubuntu Server virtual machine using Wazuh's official all-in-one installation script.

Setting a Static IP

Before installing Wazuh, a static IP address was configured on the Ubuntu Server's Host-Only network interface. This is necessary because Wazuh agents must know the fixed IP address of the manager to register and send logs. If the manager's IP changes, all agent configurations break.

The static IP was set by editing the Netplan configuration file on Ubuntu:

```
sudo nano /etc/netplan/00-installer-config.yaml
# Configured enp0s8 (Host-Only) with static IP 192.168.100.10/24
```

Installing Wazuh

Wazuh provides an official installation script that handles the entire deployment process. The script downloads, installs, and configures all three components automatically:

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

The installation process takes approximately 20–30 minutes. During this time the script configures certificates for secure communication between components, sets up the OpenSearch indexer with appropriate memory settings, creates the admin credentials for the dashboard, and starts all services automatically.

Verifying Wazuh Services

After installation, the status of all three services was verified using `systemctl`:

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

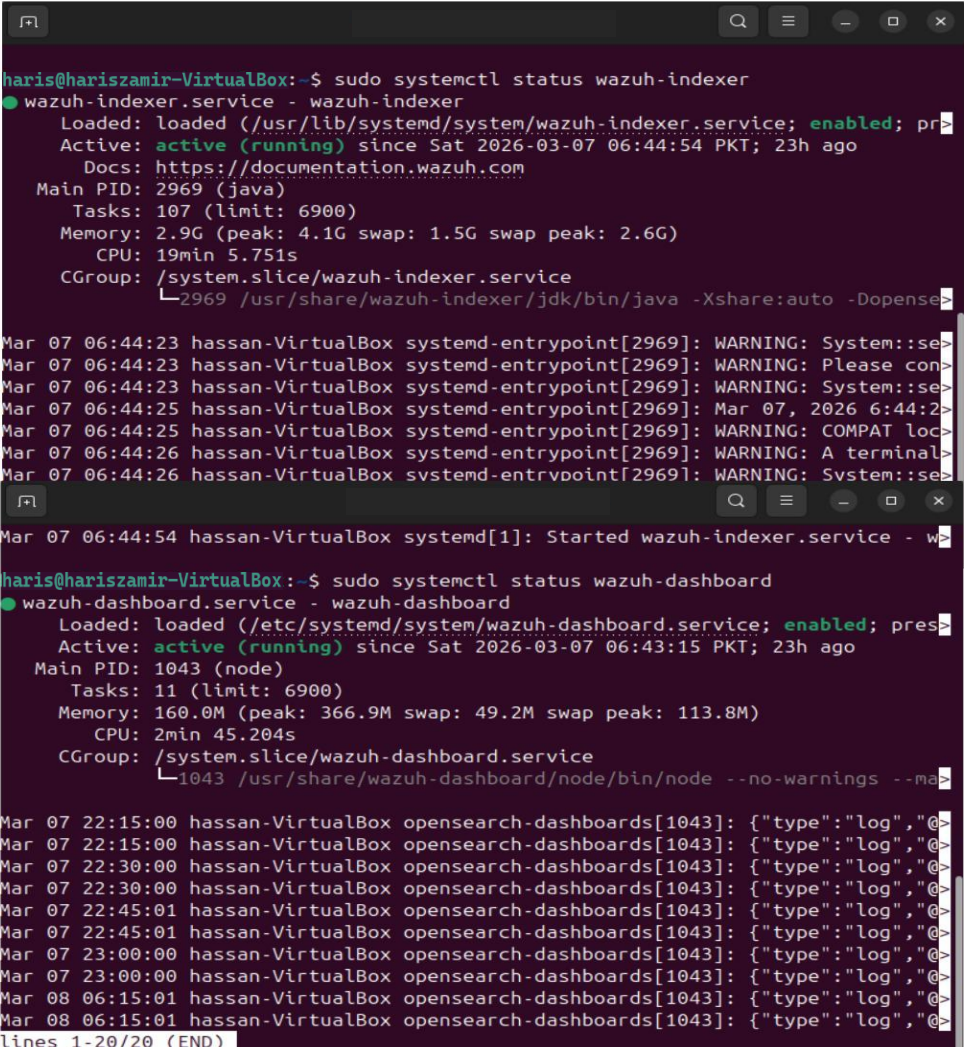
All three services reported an "active (running)" status, confirming a successful installation.

Checking Wazuh Services

After installation service status was verified.

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

All services showed **active (running)**.

A terminal window screenshot showing the status of Wazuh services. The user runs 'sudo systemctl status wazuh-indexer' and 'sudo systemctl status wazuh-dashboard'. Both services are shown as 'active (running)'. The wazuh-indexer service details include: Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled), Active: active (running) since Sat 2026-03-07 06:44:54 PKT; 23h ago, Docs: https://documentation.wazuh.com, Main PID: 2969 (java), Tasks: 107 (limit: 6900), Memory: 2.9G (peak: 4.1G swap: 1.5G swap peak: 2.6G), CPU: 19min 5.751s, CGroup: /system.slice/wazuh-indexer.service. The wazuh-dashboard service details include: Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled), Active: active (running) since Sat 2026-03-07 06:43:15 PKT; 23h ago, Main PID: 1043 (node), Tasks: 11 (limit: 6900), Memory: 160.0M (peak: 366.9M swap: 49.2M swap peak: 113.8M), CPU: 2min 45.204s, CGroup: /system.slice/wazuh-dashboard.service. There are also log messages from opensearch-dashboards[1043] showing log entries. The terminal window has a dark background with light-colored text. The prompt is 'haris@harisamir-VirtualBox:~\$'.

```
haris@harisamir-VirtualBox:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-03-07 06:44:54 PKT; 23h ago
     Docs: https://documentation.wazuh.com
   Main PID: 2969 (java)
    Tasks: 107 (limit: 6900)
   Memory: 2.9G (peak: 4.1G swap: 1.5G swap peak: 2.6G)
      CPU: 19min 5.751s
   CGroup: /system.slice/wazuh-indexer.service
           └─2969 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch

Mar 07 06:44:23 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: System::se
Mar 07 06:44:23 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: Please con
Mar 07 06:44:23 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: System::se
Mar 07 06:44:25 hassan-VirtualBox systemd-entrypoint[2969]: Mar 07, 2026 6:44:2
Mar 07 06:44:25 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: COMPAT loc
Mar 07 06:44:26 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: A terminal
Mar 07 06:44:26 hassan-VirtualBox systemd-entrypoint[2969]: WARNING: System::se

Mar 07 06:44:54 hassan-VirtualBox systemd[1]: Started wazuh-indexer.service - w

haris@harisamir-VirtualBox:~$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-03-07 06:43:15 PKT; 23h ago
     Main PID: 1043 (node)
    Tasks: 11 (limit: 6900)
   Memory: 160.0M (peak: 366.9M swap: 49.2M swap peak: 113.8M)
      CPU: 2min 45.204s
   CGroup: /system.slice/wazuh-dashboard.service
           └─1043 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --ma

Mar 07 22:15:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 22:15:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 22:30:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 22:30:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 22:45:01 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 23:00:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 07 23:00:00 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 08 06:15:01 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
Mar 08 06:15:01 hassan-VirtualBox opensearch-dashboards[1043]: {"type":"log","@
lines 1-20/20 (END)
```

Figure 7: All three Wazuh services showing active (running) status after installation

Accessing the Wazuh Dashboard

The Wazuh Dashboard was accessed through a web browser using the Ubuntu Server's Host-Only IP address:

<https://192.168.100.10>

After logging in with the admin credentials generated during installation, the dashboard displayed the Agents Summary, Last 24 Hours Alerts panel, and the full Endpoint Security and Threat Intelligence module navigation.

```
wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; pr>
Active: active (running) since Sat 2026-03-07 06:44:39 PKT; 23h ago
Process: 2970 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (co>
Tasks: 245 (limit: 6900)
Memory: 254.6M (peak: 873.4M swap: 288.3M swap peak: 343.6M)
CPU: 9min 27.304s
CGroup: /system.slice/wazuh-manager.service
├─3239 /var/ossec/framework/python/bin/python3 /var/ossec/api/scri>
├─3240 /var/ossec/framework/python/bin/python3 /var/ossec/api/scri>
├─3241 /var/ossec/framework/python/bin/python3 /var/ossec/api/scri>
├─3244 /var/ossec/framework/python/bin/python3 /var/ossec/api/scri>
├─3247 /var/ossec/framework/python/bin/python3 /var/ossec/api/scri>
├─3292 /var/ossec/bin/wazuh-authd
├─3311 /var/ossec/bin/wazuh-db
├─3348 /var/ossec/bin/wazuh-execd
├─3383 /var/ossec/bin/wazuh-analysisd
├─3410 /var/ossec/bin/wazuh-syscheckd
├─3428 /var/ossec/bin/wazuh-remoted
├─3523 /var/ossec/bin/wazuh-logcollector
├─3547 /var/ossec/bin/wazuh-monitord
└─3574 /var/ossec/bin/wazuh-modulesd
```

lines 1-23

Accessing Wazuh Dashboard

The dashboard was opened in a browser:

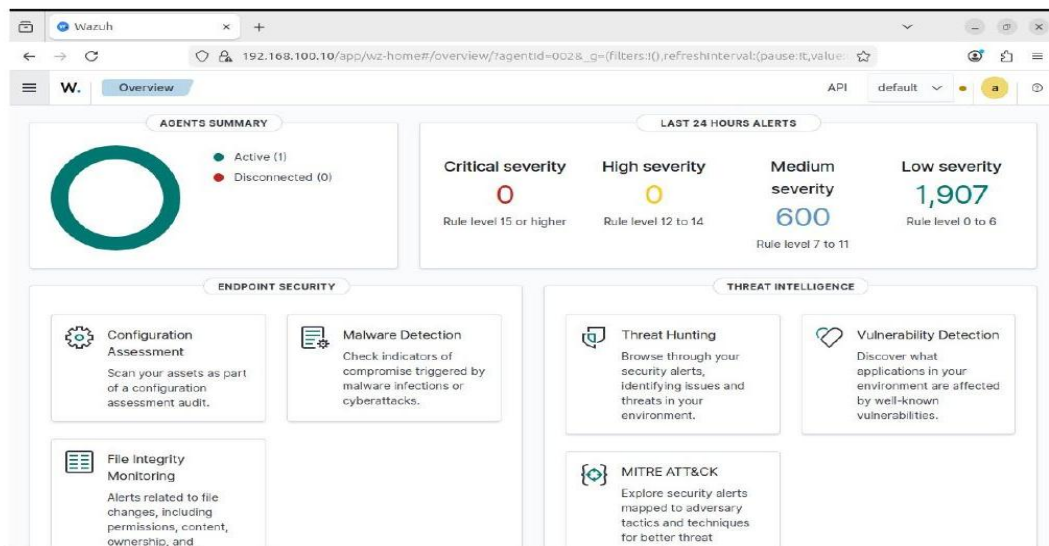


Figure 8: Wazuh Dashboard overview showing agents summary and alert severity distribution

Days 5–6 — Wazuh Agent Deployment

With the Wazuh Manager operational, the next phase involved deploying Wazuh agents on the endpoint machines. Agents are lightweight processes that collect system logs, file integrity data, and security events from an endpoint and forward them to the Wazuh Manager for analysis. This is what transforms isolated machines into actively monitored security assets.

Windows 11 Agent Installation

The Wazuh agent for Windows was installed using PowerShell. The MSI installer was downloaded directly from the official Wazuh repository:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent.msi -
OutFile wazuh-agent.msi
msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER="192.168.100.10"
net start wazuh
```

Key points about the Windows agent configuration:

- The WAZUH_MANAGER parameter in the installation command points the agent to the correct manager IP
- The agent configuration file is located at: C:\Program Files (x86)\ossec-agent\ossec.conf
- Three key configuration blocks in ossec.conf: Server block (manager IP), Syscheck (file integrity monitoring), and Localfile (monitored log paths)
- Windows Firewall must allow TCP and UDP port 1514 for agent-to-manager communication

Kali Linux Agent Installation

A Wazuh agent was also deployed on the Kali Linux machine. This serves a dual purpose: it allows monitoring of the Kali machine itself, and it provides a richer set of Linux authentication and system logs that demonstrate Wazuh's cross-platform capabilities.

```
# Download and install .deb package
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.2-
1_amd64.deb
sudo dpkg -i wazuh-agent_4.7.2-1_amd64.deb
sudo systemctl start wazuh-agent && sudo systemctl enable wazuh-agent
```

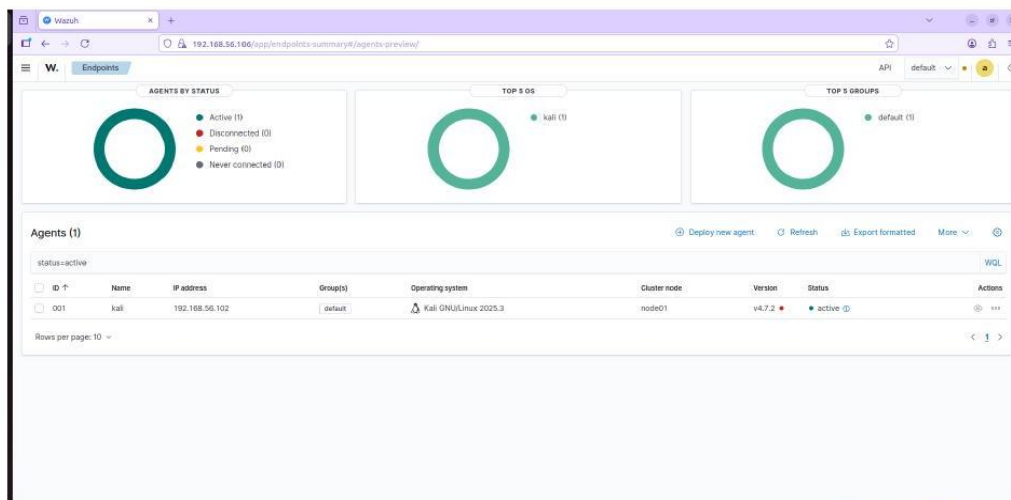
Verifying Agent Status in the Dashboard

Once both agents were started, they were verified in the Wazuh Dashboard under Endpoints → Agents. Both the Windows 11 and Kali Linux agents appeared with an "Active" status, confirming successful registration and communication with the Wazuh Manager.

Wazuh Agent Deployment on Kali Linux and Windows 10

1. Kali Linux Agent Deployment

- Downloaded the Wazuh agent `.deb` package from the official repository
- Installed the agent and fixed dependencies
- Configured the agent to connect to the Wazuh Manager by editing `/var/ossec/etc/ossec.conf` and specifying the manager IP.
- Started the agent and enabled auto-start on boot:
- **Observation:** The Kali agent temporarily appeared disconnected when the VM was shut down to reduce host resource usage. Once restarted, it automatically reconnected.



2. Windows 10 Agent Deployment

- Downloaded the official Windows MSI installer:
<https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.2-1.msi>
- Installed using GUI and entered the **Wazuh Manager IP**.
- **Issue encountered:** The agent service initially failed to start, showing “The service did not report an error.”
- **Resolutions applied:**
 1. Verified the `<server>` section in `ossec.conf` had the correct manager IP.
 2. Configured Windows Firewall to allow TCP and UDP **port 1514**.
 3. Installed **Visual C++ Redistributable** (if missing).
 4. Started the service using the correct service name

Figure 9: Wazuh Dashboard showing both Kali and Windows agents with Active status

Generating Test Alerts

To verify that the monitoring pipeline was working end-to-end, test alert conditions were deliberately triggered. Failed SSH login attempts were generated from the Kali Linux machine targeting the Ubuntu Server:

```
ssh wronguser@192.168.100.10
```

Multiple incorrect password attempts were entered deliberately. Each failed attempt generated a log entry on the Ubuntu Server, which was forwarded to the Wazuh Manager and processed against detection rules. The resulting alerts appeared in the Wazuh Dashboard within seconds, confirming the complete monitoring pipeline was functional.

Day 7 — Alert Triage and Analysis

The final day of Week 1 was dedicated to alert triage — the process of reviewing, categorizing, and prioritizing security alerts. This is one of the most critical skills a SOC analyst must develop. Not every alert represents a genuine threat; a significant portion of alerts in any SOC environment are false positives, informational events, or expected system behaviors. The ability to quickly and accurately distinguish signal from noise is what separates an effective SOC from a reactive one.

Wazuh Alert Severity Levels

Wazuh assigns a numeric rule level (0–15) to every alert based on the severity of the detected event. Understanding these levels helps analysts prioritize their response:

Rule Level	Classification	Typical Response
0 – 3	Informational	Log for reference; no action required
4 – 7	Low Severity	Monitor for patterns; investigate if repeated
8 – 11	Medium Severity	Investigate promptly; may indicate a real threat
12 – 15	High / Critical	Immediate investigation required; possible active attack

Threat Hunting in Wazuh

The Threat Hunting module in Wazuh provides a powerful event search interface. Analysts can filter events by agent name, rule level, rule ID, timestamp, or any field in the event data. This enables focused investigation of specific time windows or specific machines, rather than reviewing all alerts indiscriminately.

Filters applied during this lab session:

- `agent.name` — to isolate events from a specific VM
- `rule.level` — to filter by severity (e.g., only Medium and above)
- `rule.id` — to investigate specific alert types
- Time range — Last 15 minutes / Last 24 hours

Investigating Alerts

Alerts were reviewed inside the **Security Events** section of the dashboard.

Each alert includes:

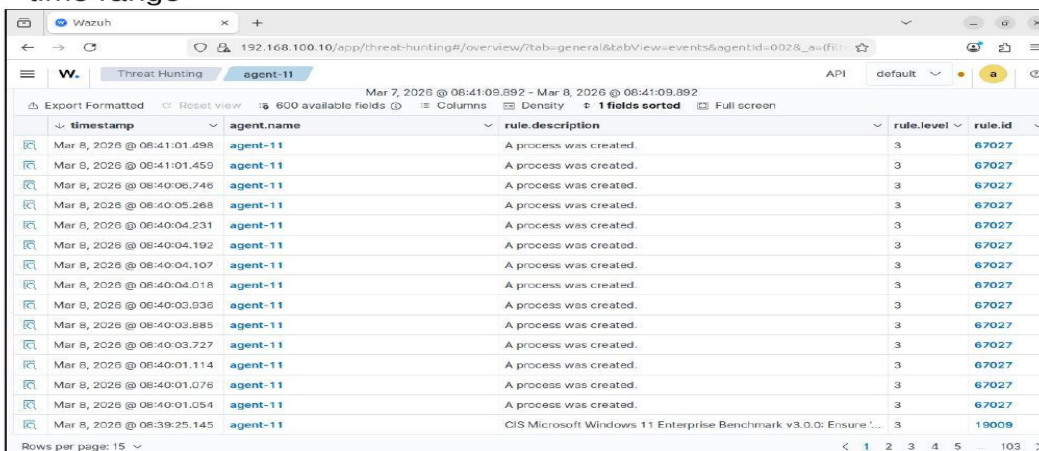
- Rule ID
- Severity level
- Agent name
- Description
- Timestamp

Threat Hunting

The Threat Hunting module allows filtering events.

Example filters:

- agent.name
- rule.level
- rule.id
- time range



The screenshot shows the Wazuh Threat Hunting module interface. The table displays filtered security events for agent-11. The columns are: timestamp, agent.name, rule.description, rule.level, and rule.id. The events are sorted by rule.id in descending order. The first 15 rows are shown, with a total of 103 rows. The last row is highlighted in blue.

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 8, 2025 @ 06:41:01.498	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:41:01.459	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:06.746	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:05.268	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:04.231	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:04.192	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:04.107	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:04.018	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:03.936	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:03.889	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:03.727	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:01.114	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:01.076	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:40:01.054	agent-11	A process was created.	3	67027
Mar 8, 2025 @ 06:39:25.145	agent-11	CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0: Ensure...	3	19009

24

Figure 10: Wazuh Threat Hunting module displaying filtered security events with rule levels and IDs

MITRE ATT&CK Mapping

One of Wazuh's most powerful features is its integration with the MITRE ATT&CK framework. Every alert that Wazuh generates is automatically mapped to the corresponding ATT&CK technique where applicable. This allows analysts to immediately understand the context of an alert — not just what happened, but what phase of an attack it corresponds to.

Example mappings observed during this lab:

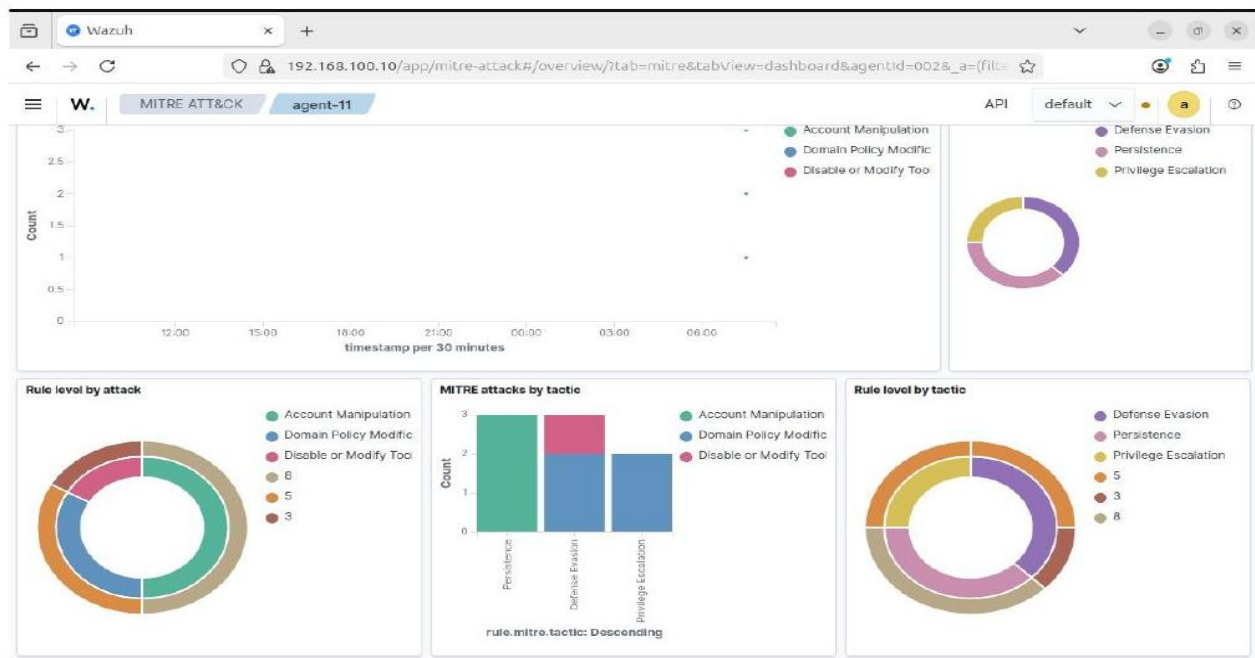
- Failed SSH login attempts → T1110: Brute Force (Credential Access tactic)
- Sudo usage on Kali → T1548: Abuse Elevation Control Mechanism (Privilege Escalation)
- Windows service modifications → T1543: Create or Modify System Process (Persistence)
- System tool modifications → T1562: Impair Defenses (Defense Evasion)

MITRE ATT&CK Mapping

Wazuh maps alerts to the **MITRE ATT&CK framework**, which categorizes attacker techniques.

Example:

Brute-force login attempts map to **Credential Access techniques**.



Alert Triage Analysis

During normal system activity and testing, several Wazuh alerts were generated. These alerts represent different types of system events detected by Wazuh monitoring rules.

The most frequently observed alerts included:

- 52002
- 60642

During the lab session, a variety of alerts were generated through both normal system activity and deliberate attack simulation. The following table summarizes the most significant alert rules observed and their analyst assessment:

Rule ID	Level	Description	Trigger	Assessment
52002	5 (Med)	Authentication failure	Multiple wrong SSH passwords from Kali	Expected — deliberate brute-force test
60642	3 (Low)	Windows event log activity	Normal Windows system logging	Benign — monitoring pipeline working
504	3 (Low)	SSH login attempt	SSH connection initiated	Normal — part of test scenario
750	4 (Low)	SSH authentication failure	Incorrect password entered	Generated during brute-force testing
61104	3 (Low)	Wazuh agent event	Agent communicating with manager	Normal — healthy monitoring activity
19012	2 (Info)	Windows event monitoring	Windows security log collection	Expected — agent collecting logs
19005	2 (Info)	Windows log detected	Standard Windows event log	Expected monitoring result

Detailed Rule Analysis

Rule 60642 — Windows Event Log Activity

This alert indicates that the Wazuh agent successfully detected and forwarded entries from the Windows Event Log. Windows continuously records system events including service starts and stops, user logon events, application errors, and policy changes. In this lab context, Rule 60642 alerts were largely informational — they confirmed that the Windows monitoring pipeline was fully operational. In a production SOC environment, this rule's events would be baselined and reviewed for anomalies rather than treated as individual threats.

Rules 504 and 750 — SSH Authentication

These two rules together tell a coherent story about SSH activity. Rule 504 fires when an SSH connection is initiated, and Rule 750 fires when authentication fails. During the brute-force simulation from Kali Linux, both rules fired repeatedly in sequence — exactly the pattern a SOC analyst would look for when investigating a potential brute-force attack. A single instance of Rule 750 is low-severity. A sustained series of 504 → 750 sequences from a single source IP is a high-confidence indicator of brute-force activity.

Rule 61104 — Agent Communication

This rule confirms that the Wazuh agent is actively communicating with the manager — essentially a heartbeat confirmation. From an operational standpoint, Rule 61104 is useful as a health check. If this alert suddenly stops appearing for a particular agent, it may indicate the agent has crashed, the endpoint is unreachable, or — in adversarial scenarios — an attacker may have terminated the monitoring process.

Rules 19012 and 19005 — Windows Log Monitoring

These informational rules confirm that the Wazuh agent on the Windows 11 machine is successfully collecting from all three primary Windows event log channels: Security logs (logon events, privilege use), System logs (service events, hardware events), and Application logs (software-specific events).

Their presence in the dashboard is a positive confirmation that the Windows endpoint is being fully monitored.

SOC Analyst Perspective and Decision Process

A core competency in SOC work is developing sound judgment about when to escalate versus when to dismiss. The following framework was applied during this lab's triage exercise:

1. Identify the rule ID and its severity level
2. Determine which agent and which endpoint generated the alert
3. Review the full log message and event details for context
4. Look for correlated events in the same time window (e.g., multiple 750s following a 504)
5. Assess frequency — a one-off event vs. a sustained pattern changes the threat level entirely
6. Apply context — is this during business hours? From a known IP? During a known maintenance window?
7. Make a triage decision: Close as expected behavior / Monitor for recurrence / Escalate for investigation

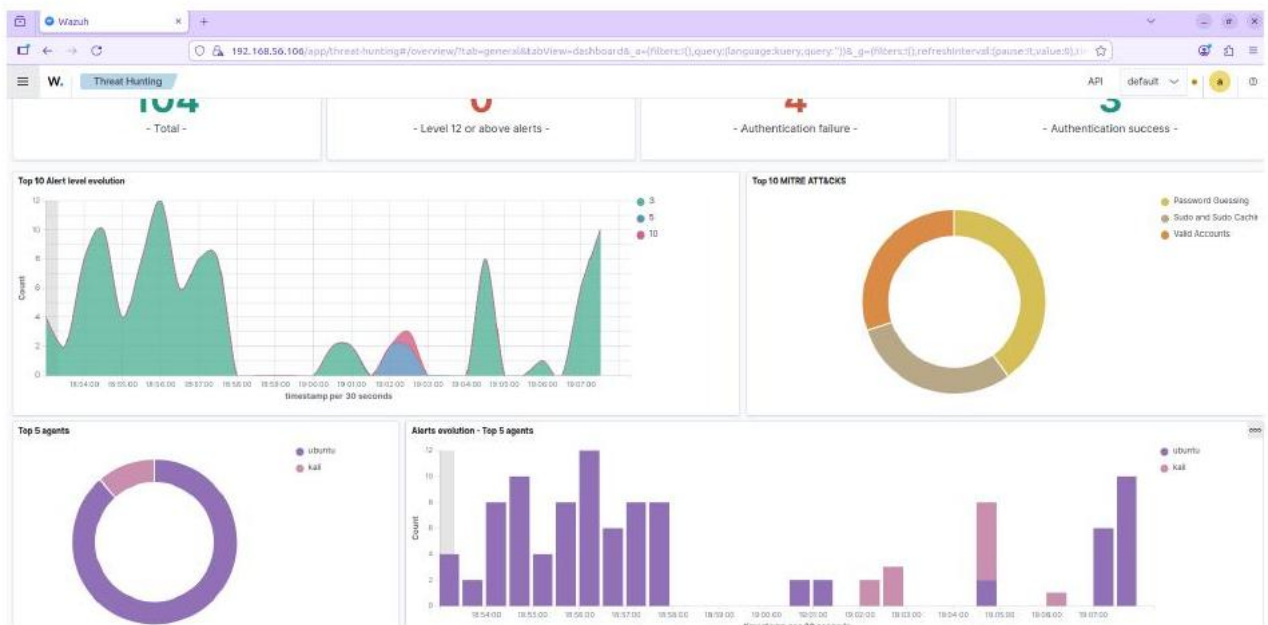
Applied to this lab: The SSH authentication failures (Rule 750) were individually low-severity but the pattern — multiple sequential failures from Kali's IP within a short time window — would warrant escalation in a production environment. An analyst would document the source IP, check for other activity from that IP, and potentially initiate an incident response workflow.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ touch /tmp/testfile.txt
echo "This is a test modification" >> /tmp/testfile.txt

(kali@kali)-[~]
$ systemctl status cron
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-03-07 13:36:16 EST; 27min ago
  Invocation: 55138249d8ee4d00b86c775f580a5707
     Docs: man:cron(8)
    Main PID: 495 (cron)
      Tasks: 1 (limit: 4456)
     Memory: 572K (peak: 2.1M)
        CPU: 237ms
    CGroup: /system.slice/cron.service
            └─495 /usr/sbin/cron -f

Mar 07 13:36:16 kali cron[495]: (CRON) INFO (Running @reboot jobs)
Mar 07 13:39:01 kali CRON[2854]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Mar 07 13:39:01 kali CRON[2856]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/session
Mar 07 13:39:01 kali CRON[2854]: pam_unix(cron:session): session closed for user root
Mar 07 13:45:01 kali CRON[2956]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Mar 07 13:45:01 kali CRON[2958]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Mar 07 13:45:02 kali CRON[2956]: pam_unix(cron:session): session closed for user root
Mar 07 13:55:01 kali CRON[3005]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Mar 07 13:55:01 kali CRON[3007]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Mar 07 13:55:01 kali CRON[3005]: pam_unix(cron:session): session closed for user root

(kali@kali)-[~]
$ sudo systemctl stop cron
[sudo] password for kali:
```



This image shows all the attacks being detected by wazuh.

Figure 11: Wazuh Dashboard showing all detected attacks from both agents during lab testing

Conclusion

Week 1 of the Cyberster Blue Team SOC Internship delivered a comprehensive and practical introduction to the core skills required for SOC operations. Starting from first principles — understanding how data moves through OSI and TCP/IP layers — the week progressed logically through each stage of building a functional security monitoring environment.

The VirtualBox lab environment successfully simulated a minimal enterprise network, with a Wazuh SIEM platform monitoring two different endpoint operating systems. The deliberate generation of test alerts through SSH brute-force simulation demonstrated the complete monitoring pipeline: from attacker action → endpoint log collection → SIEM analysis → dashboard alert → analyst triage.

Key Learnings

- Networking foundations (OSI model, TCP/IP, subnetting) are not just theoretical — they directly inform how SOC analysts read and interpret security alerts
- Wazuh provides a powerful, extensible SIEM platform that integrates seamlessly with the MITRE ATT&CK framework
- Alert triage is as much an art as a science — context, patterns, and analyst judgment are as important as the raw alert data
- A single failed SSH login is noise; hundreds of failures from a single IP in a short window is a signal
- The monitoring pipeline (Agent → Manager → Indexer → Dashboard) must be validated end-to-end before any meaningful detection work can begin

This lab environment provides the foundation for future exercises involving more sophisticated threat scenarios — including malware analysis, lateral movement detection, file integrity monitoring, and advanced threat hunting using custom detection rules.